

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

ОТЧЕТ
о лабораторном практикуме

По дисциплине DevTestOps

На тему Настройка ILM и Pipeline

Выполнил обучающийся:

Груздев Николай Александрович
(Ф.И.О.)

Направление подготовки / специальность:

09.03.01 Информатика и вычислительная
техника
(код и наименование)

Курс: 4

Группа: 151220

Руководитель: Тарасов А.П., старший
преподаватель кафедры информационных
систем и информационной безопасности САФУ
(Ф.И.О. руководителя, должность / уч. степень / звание)

Отметка о зачете

(отметка прописью)

(дата)

Руководитель

(подпись руководителя)

А.П. Тарасов
(инициалы, фамилия)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛЬ РАБОТЫ

Создать и применить ILM политику для логов nginx, настроить парсинг логов grok при помощи pipeline.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Для автоматизированной обработки журналов веб-сервера для начала будет произведена остановка парсинга логов в logstash путем закомментирования раздела filter как указано на рисунке 1. После будет сконфигурирован встроенный конвейер (ingest pipeline). В веб-консоли Kibana иницилируем создание нового пайплайна с идентификатором Nginx-logs-parser. Первым этапом конфигурации станет добавление процессора grok, в рамках которого задаются два вспомогательных обработчика типа set, предназначенные для корректной маркировки сценариев с ошибками парсинга (рисунки 1–3).

```
#filter {
#  if [app] == "nginx" {
#    grok {
#      match => { "message" => "%{IP:client_ip} - - [{HTTPDATE:timestamp}]"
#    }
#    date {
#      match => ["timestamp", "dd/MMM/yyyy:HH:mm:ss Z"]
#      target => "@timestamp"
#      remove_field => ["timestamp"]
#    }
#    mutate {
#      add_field => { "from_host" => "vm3" }
#    }
#    geoip {
#      source => "client_ip"
#      target => "geoip"
#    }
#  }
#}
```

Рисунок 1 – Прекращение парсинга

Manage processor [Grok documentation](#)

Configuration Output

Processor: Grok

Uses grok expressions to extract matches from a field.

Field: message

Field to search for matches.

Patterns
Grok expressions used to match and extract named capture groups. Uses the first matching expression.

= %{IPORHOST:client_ip} - %{NOTSPACE:user.name} \[%{HTTPDATE:nginx.access.time}\]

[Add pattern](#)

Рисунок 1 – Настройка процессора Grok в конвейере обработки

Manage on-failure processor Set documentation

Configuration **Output**

Processor
 Set
 Sets the value of a field.

Field
 error.message
 Field to insert or update.

Value Define as JSON
 Grok parsing failed for Nginx log
 Value for the field.

☒ **Use Copy instead of Value**
 Specify fields to copy into **Field** instead of setting a **Value**.

☒ **Override**
 If enabled, overwrite existing field values. If disabled, only update **null** fields.

Рисунок 2 – Конфигурация первого вспомогательного обработчика Set

Manage on-failure processor Set documentation

Configuration **Output**

Processor
 Set
 Sets the value of a field.

Field
 error.type
 Field to insert or update.

Value Define as JSON
 nginx_grok_failure
 Value for the field.

☒ **Use Copy instead of Value**
 Specify fields to copy into **Field** instead of setting a **Value**.

☒ **Override**
 If enabled, overwrite existing field values. If disabled, only update **null** fields.

Рисунок 3 – Параметры второго обработчика Set для обработки ошибок

Следующим шагом добавим процессор `date`, который обеспечит преобразование временной метки из исходного лога в стандартное поле `@timestamp`, используемое Elasticsearch для хронологической сортировки (рисунок 4).

Manage processor

[Date documentation](#)

Configuration

Output

Processor

Date

Converts a date to a document timestamp.

Field

nginx.access.time

Field to convert.

Formats

dd/MMM/yyyy:HH:mm:ss Z

Expected date formats. Provided formats are applied sequentially. Accepts a Java time pattern or one of the following formats: `ISO8601`, `UNIX`, `UNIX_MS`, `TAI64N`.

Target field (optional)

@timestamp

Output field. If empty, the input field is updated in place. Defaults to `@timestamp`.

Timezone (optional)

UTC

Timezone for the date. Defaults to `UTC`.

Locale (optional)

Cancel

Update

Рисунок 4 – Настройка процессора Date для парсинга временной метки

Для оптимизации хранимых данных воспользуемся процессором `remove`, исключаящим из итогового документа избыточное поле `nginx.access.time` (рисунок 5).

Manage processor Remove documentation

Configuration **Output**

Processor

Remove

Removes one or more fields.

Fields

nginx.access.time

Fields to remove.

☒ Ignore missing

Ignore documents with a missing field.

Condition (optional)

An if condition written as a Painless script: `ctx?.network?.name == 'Guest'`

Tag (optional)

An identifier for the processor. Useful for debugging and metrics.

☒ Ignore failures for this processor

Cancel Update

Рисунок 5 – Конфигурация процессора удаления дублирующего поля

Дополнительно внедрим ещё один процессор типа `set`, который будет проставлять служебное поле `pipelined` со значением `yes`, сигнализирующее об успешном прохождении записи через конвейер (рисунок 6).

Manage processor Set documentation

Configuration **Output**

Processor

Set

Sets the value of a field.

Field

pipelined

Field to insert or update.

Value

yes

Value for the field.

☒ Use Copy instead of Value

Specify fields to copy into Field instead of setting a Value.

☒ Override

If enabled, overwrite existing field values. If disabled, only update null fields.

Update

Рисунок 6 – Добавление маркера успешной обработки в ingest pipeline

Завершающим элементом конвейера станет процессор `geoip`, осуществляющий обогащение записей географической информацией на основе значения поля `client_ip` (рисунок 7).

Manage processor [GeolIP documentation](#)

Configuration Output

Processor
GeolIP
Adds geo data based on an IP address. Uses geo data from a MaxMind database file.

Field
client.ip
Field containing an IP address for the geographical lookup.

Target field (optional)
client.geo
Field used to contain geo data properties.

Database file (optional)

GeolIP2 database file in the `ingest-geoip` configuration directory. Defaults to `GeoLite2-City.mmdb`.

Properties (optional)
Type and then hit "ENTER"
Properties added to the target field. Valid properties depend on the database file used.

☒ First only
Use the first matching geo data, even if the field contains an array.

☒ Ignore missing

Рисунок 7 – Настройка геокодирования по IP-адресу клиента

Итоговая структура сконфигурированного конвейера обработки данных представлена на рисунке 8.

Processors [Import processors](#) [Test pipeline: Add documents](#)

Use processors to transform data before indexing. [Learn more.](#)

Grok Extracts values from "message" that match a grok pattern.

Failure handlers

- Set** Sets value of "error.message" to "Grok parsing failed for Nginx log".
- Set** Sets value of "error.type" to "nginx_grok_failure".

[Add a processor](#)

GeolIP Adds geo data to documents based on the value of "client.ip".

Date Parses a date from "nginx.access.time" to a date type on field "@timestamp".

Set Sets value of "pipelined" to "yes".

Remove Removes "nginx.access.time".

Рисунок 8 – Полная схема пайплайна Nginx-logs-parser

Далее перейдём к настройке политики управления жизненным циклом индексов (ILM). В фазах `warm` и `cold` определим временные интервалы (в днях), по истечении которых индексы будут автоматически переведены на соответствующие уровни хранения. Для фазы `cold` дополнительно активируем режим «только чтение», запрещающий модификацию архивных данных (рисунки 9 и 10).

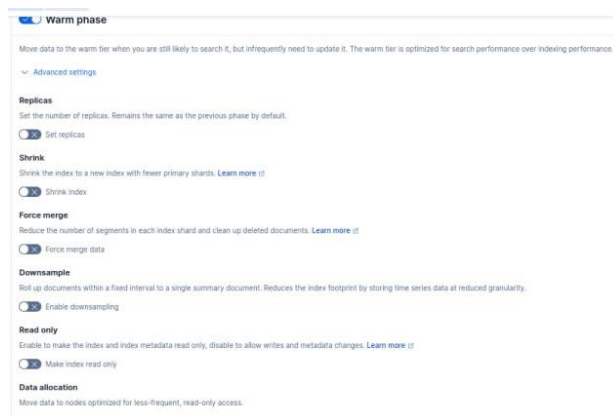


Рисунок 9 – Параметры перехода индексов в фазу Warm

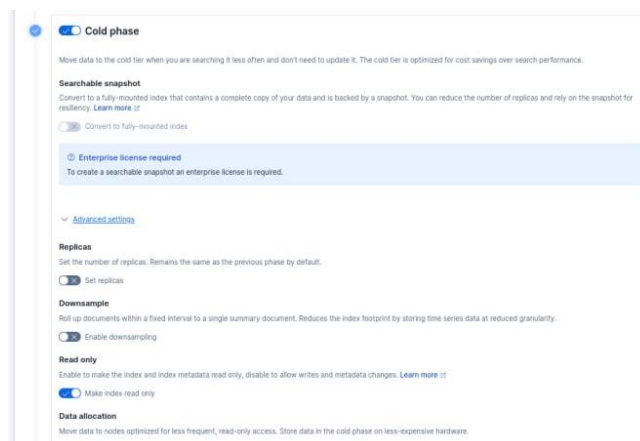


Рисунок 10 – Конфигурация фазы Cold с ограничением на запись

В заключительной фазе delete зададим срок автоматической очистки устаревших индексов из кластера (рисунок 11).

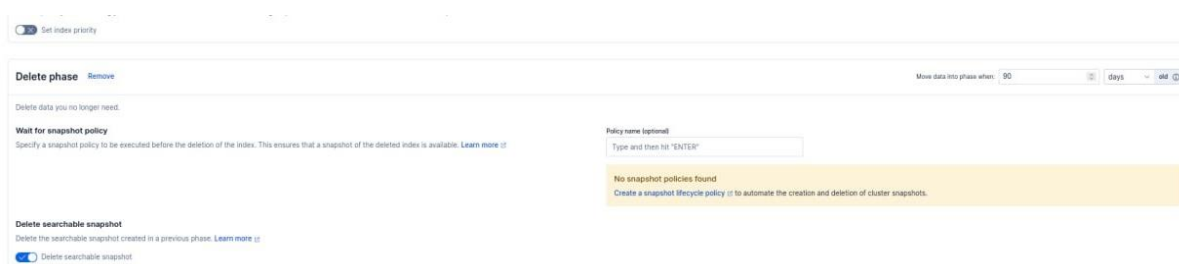


Рисунок 11 – Настройка периода хранения перед удалением данных

После сохранения конфигурации ILM привяжем созданную политику и ingest-пайплайн к шаблону индексов nginx-template. Для этого в параметрах шаблона скорректируем настройки индексации, явно указав ссылки на соответствующие компоненты (рисунок 12).

```

Index settings
{
  "index": {
    "lifecycle": {
      "name": "Nginx-logs-ilm",
      "rollover_alias": "nginx-logs"
    },
    "mode": "standard",
    "routing": {
      "allocation": {
        "include": {
          "_tier_preference": "data_content"
        }
      }
    },
    "default_pipeline": "Nginx-logs-parser",
    "number_of_shards": "3",
    "number_of_replicas": "1"
  }
}

```

Рисунок 12 – Интеграция ILM-политики и пайплайна в шаблон индекса

Зафиксируем изменения в шаблоне и отредактируем конфигурационный файл `output.conf` агента Logstash, перенаправив выходной поток данных в целевой индекс `nginx-logs` (рисунок 13).

```

} else {
  elasticsearch {
    hosts => ["https://elk.elastic.com:9200"]
    index => "nginx-logs"
    data_stream => false
    ssl_certificate_authorities => '/etc/logstash/certs/ca.crt'
    user => "elastic"
    password => "EPiWxnR4dQYvg-TU53q5"
  }
}

```

Рисунок 13 – Модификация конфигурации вывода Logstash для целевого индекса

ВЫВОД

В рамках выполнения лабораторной работы был разработан и внедрён механизм автоматической обработки журналов Nginx с использованием ingest pipeline, включающий структурный парсинг через модуль Grok и обогащение записей географическими атрибутами. Дополнительно реализована политика управления жизненным циклом индексов (ILM), обеспечивающая автоматический переход данных между фазами хранения, перевод архивов в режим «только чтение» и их своевременную очистку в соответствии с заданными временными параметрами.